Automated Documentation Drift Detection

All points from the provided document, rewritten in easier language

What this topic is about: This project aims to find places where a project's code has changed but its written documentation may no longer match the code.

---

1. The five questions asked

1. Why does this problem happen?

2. What is the easiest way to solve this problem?

3. How can a user access the solution?

4. Explain every 'why' and 'how', especially the reasons behind the problem.

5. How much time, effort and money will it take to build and run this solution?

2. Why does the problem happen?

2.1 The connection between code and documentation is not clear (implicit coupling).

The code and its documentation are connected, but that connection is usually not clearly recorded. When someone changes the code, they may not know which document sentences also need to change.

2.2 The problem is growing in size.

As projects become larger and receive more code changes, there is more documentation to check. It becomes harder for people to notice every place where the written explanation has become outdated.

2.3 The cost of documentation drift is unequal (asymmetric).

A small code change can make a document incorrect. Finding and fixing that mismatch later may take much more effort than making the original code change. The cost of missing the mismatch can be much higher than the cost of the change itself.

In simple words: code can change quickly, but the related documentation may be missed. That difference creates documentation drift.

3. Easiest way to solve the problem

The proposed process checks code changes in a GitHub Pull Request (PR), compares them with the documentation, and reports possible mismatches.

Overall process

Step	Meaning in easy language
GitHub PR	A developer submits code changes for review.
Diff	The system checks what was added, removed or changed in the code.
Extract facts	It picks out facts from the changed code that can be checked.
Find documentation candidates	It searches for document sentences that may describe those facts.
LLM truth-check	An AI language model checks whether each selected sentence still matches the code.
PR comment	**The system reports its findings in the Pull Request.**

The five stages

Stage 1 – Trigger and diff

Start the check when a GitHub Pull Request is opened or updated. Compare the new code with the earlier code to see what changed.

Stage 2 – Extract checkable facts from changed code

Read the changed code and identify facts that can be checked, such as changed names, values, behaviour or rules.

Stage 3 – Find candidate documentation sentences

Search the documentation for sentences that might describe the changed code.

Stage 4 – AI truth-checks each candidate sentence

Ask the AI model to compare each selected documentation sentence with the code and decide whether they still agree.

Stage 5 – Report on the Pull Request

Show the possible documentation problems as comments on the Pull Request so the developer can review them.

4. How can the user access the solution?

The document lists three ways to use the tool. Each option is meant for a different type of user or team.

Access method	Who uses it	How it works (easy language)
GitHub App (main option)	A maintainer of any project	Install the app from the GitHub Marketplace and give it permission to access the repository. Future Pull Requests are checked automatically and receive comments about possible documentation drift. No code or repository configuration is needed.
GitHub Action (self-hosted)	Teams that do not want a third-party app	Add a small workflow file named `.github/workflows/docdrift.yml`. It calls the tool and uses an `OPENAI_API_KEY` secret. The check runs inside the team's own CI system.
CLI (local / offline)	**Contributors before opening a Pull Request**	**Install it with `pip install docdrift`, then run `docdrift check --base main`. It runs on the contributor's computer and prints findings in the terminal. This helps find drift before creating a Pull Request.**

5. Every 'why' and 'how' question

The original document lists the following questions to be investigated. They are kept in the same order and explained in simpler words. The source document lists these as questions; it does not provide detailed answers for each one.

4.a How to connect documentation sentences to source code

How can the tool work out which exact parts of the code are explained by a particular sentence in the documentation?

4.b How to identify indirect relationships

Sometimes a code change affects another part of the project without changing it directly. How can the tool find these less obvious connections?

4.c How to minimize false alarms

How can the tool avoid warning users when the documentation is actually correct? A false alarm is a warning about a problem that is not really there.

4.d How to distinguish documentation drift from changes that genuinely require documentation updates

How can the tool tell the difference between an outdated sentence and a code change that does not need any documentation change?

4.e How can the tool remain useful without excessive warnings?

How can the tool show important problems without sending so many warnings that users start ignoring them?

6. Amount of work needed to build it (effort and time)

The source document divides the work into five phases. The time estimates below are retained from the source and explained in simple language. They are estimates, not guaranteed completion dates.

Phase	Work included (easy language)	Time for 1 developer	Time for 2 developers
P0 – Skeleton	Set up the basic GitHub Action, collect code differences, and create a simple fact extractor using patterns/regular expressions.	2–3 days	1–2 days
P1 – Core detection	Extract facts from code, create an index of documentation, match code and documentation using words and meaning, and run the AI checking loop.	5–7 days	3–4 days
P2 – GitHub integration	Add Pull Request comments, a configuration file, warning thresholds and duplicate-warning prevention.	2–3 days	1–2 days
P3 – Evaluation harness	Test the tool on old project histories, check whether it can find drift before documentation is fixed, and measure precision and recall (how accurately it finds real issues and avoids missing them).	3–4 days	2 days
P4 – Polish	Improve the command-line tool (CLI), dashboard, commands for muting warnings, and documentation.	2–3 days	1–2 days
Total	**Estimated total development effort.**	**About 14–20 person-days**	**About 8–10 person-days (around 1.5–2 weeks)**

7. Estimated running cost

The source gives approximate costs for running the tool. These amounts depend on the assumptions shown in the table. They are estimates, not fixed prices.

Item	Assumption used	Cost for one PR	Monthly cost (50 PRs)
Embeddings (document index, built once per release)	About $0.02 to build a full index	About $0.001	Less than $1
AI truth-checks (around 30 candidates × around 600 tokens, using a small model)	About $0.15–$0.60 per 1 million input tokens	$0.01–$0.05	$0.50–$2.50
GitHub Actions compute	Around 2 minutes per PR on the free tier	Free for public repositories	Free
Hosting (GitHub App mode)	Serverless hosting / Fly.io hobby tier	Not specified	$0–$10
Total	**Estimated total running cost**	**About $0.01–$0.06 per PR**	**About $1–$13 per month**

How to understand the cost: Embeddings help the tool search documentation by meaning. The AI truth-check cost comes from asking the language model to review possible matches. GitHub Actions may be free for public repositories under the stated assumption. Hosting may add a monthly charge when using the GitHub App.

8. Why this problem matters: GitHub activity

The source document quotes GitHub's Octoverse 2024 report: 5.2 billion contributions across 518 million projects, with 1.4 million new developers joining open source in one year.

In easy language: a very large amount of code is being changed across many projects. Documentation may also be written or updated by someone different from the person changing the code. Because of this, it is easier for the code and documentation to stop matching.

Source mentioned in the original document: GitHub Blog, Octoverse 2024.

9. Quick revision

Main idea: When code changes, the related documentation should also be checked. This project proposes an automated tool that checks code changes in a GitHub Pull Request, finds related documentation, uses an AI model to compare them, and reports possible mismatches.

Ways to use it: GitHub App, GitHub Action, or local command-line tool (CLI).

Estimated build time: 14–20 person-days for one developer, or about 8–10 person-days for two developers.

Estimated running cost: about $0.01–$0.06 per PR, or about $1–$13 per month for 50 PRs, based on the source document's assumptions.

Note: The wording has been simplified, while the original points, stages, access methods, phase estimates, cost estimates and figures have been retained. Where the original document only listed a question, this PDF explains what that question means instead of inventing an answer.